

Image Segmentation:-

Let R represent the entire spatial region occupied by an image. We may view image segmentation, as a process that partitions R into n subregions, $R_1, R_2, \dots, R_n$ such that

(i) $\bigcup_{i=1}^n R_i = R$ [Completeness] (Every pixel must be in a region) [Connectedness]

(ii) R_i is a connected set for $i=0, 1, 2, \dots, n$
(All the pixels in a region are connected)

(iii) $R_i \cap R_j = \emptyset \quad \forall i \text{ and } j \text{ where } i \neq j$ [Disjointness]

(iv) $Q(R_i) = \text{TRUE}$ for $i=0, 1, 2, \dots, n$ [Homogeneity]

(v) $Q(R_i \cup R_j) = \text{FALSE}$ for any adjacent region R_i and R_j
[Different]
 Q can be any property for the region. R_i and R_j are said to be adjacent if their union forms a connected set, if not then disjoint.

Image Segmentation

Discontinuity Based

(Boundaries of regions are sufficiently different from each other and from the background)

[Edge, Line, Isolated Points]
Edge Based Segmentation

Similarity/Homogeneity Based

(Provide regions with similar Properties)

[Region Based Segmentation]

(a) **Edge** :- Edge pixels are pixels at which the intensity of an image changes abruptly and edges are sets of connected edge pixels.

Lines :- It can be viewed as a thin edge segment in which the intensity of the background on either side of it is either much higher or much lower than the intensity of the line pixels.

Isolated Points :- A foreground (background) pixel surrounded by background (foreground) pixels.

Isolated Point Detection :-

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

1	1	1
1	-8	1
1	1	1

$$= f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

A point has been detected at a location (x, y) on which the kernel is centered if the absolute value of the response of the filter at that point exceeds a specified threshold.

Let's say $g(x, y)$ is the output image, T is a nonnegative threshold.

$$g(x, y) = \begin{cases} 1 & \text{if } |\nabla^2 f(x, y)| > T \\ 0 & \text{otherwise.} \end{cases}$$

Line Detection :-

The Laplacian detector kernel is isotropic, so its response is independent of direction, often interest lies in detecting lines in specified directions.

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal

2	-1	-1
-1	2	-1
-1	-1	2

+45°

-1	2	-1
-1	2	-1
-1	2	-1

Vertical

-1	-1	2
-1	2	-1
2	-1	-1

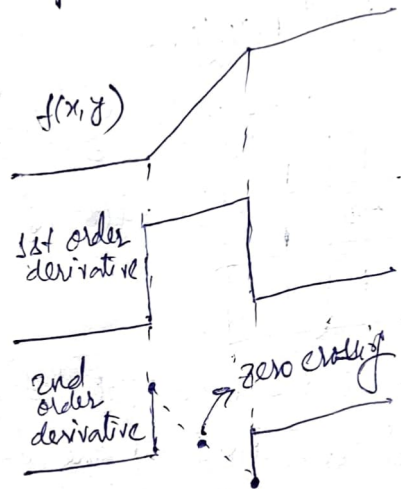
-45°

Suppose, z_1, z_2, z_3, z_4 denotes the responses of the kernels. If at a given point in the image $|z_k| > |z_j|, \forall j \neq k$, that point is said to be more likely associated with a line in the direction of kernel k .

Suppose $|z_s| > |z_j|$ for $j = 2, 3, 4$ that point is said to be most likely associated with a horizontal line. To detect all the lines in an image, sum the kernel through the image and threshold the absolute value of the result. The nonzero points remaining after thresholding are the strongest responses which, for lines one pixel thick, correspond closest to the direction defined by the kernel.

Edge Models

- (a) Step Edge - A step edge is characterized by a transition between two intensity levels occurring ideally over the distance of one pixel (No blurring)



- (b) Ramp - This comes up due to noise and blurring. The slope of the ramp is inversely proportional to the degree to which the edge is blurred.

In 1st order derivative,

- (a) Produce thick edges

- (b) In 2nd order derivative, response for the isolated noise points is much higher. → produce much thinner edges.

- (a) Roof - Roof edges are models of lines through a region, with the base of the edge being determined by the thickness and sharpness of the line; it arises when thin objects are closer to the sensor than the background.

In both ramp and step edges, the 2nd order derivative has opposite signs, provides whether we move from (+ve) low to high intensity or reverse (-ve), i.e. produces double-edge response at ramp and step transitions in intensity.

$$\text{Response } Z = \sum_{k=-1}^1 w_k z_k$$

to the filter $k=1$
in 2nd derivative,

zero crossing can be used for locating the center of thick edges produces two values for every edge in a image.

Steps in Edge Detection:

① Image smoothing for noise reduction/suppression.

② Detection of edge points [extracts all points that are potential edge-point candidates]

③ Edge localization [To select the edgepoints from the candidates]

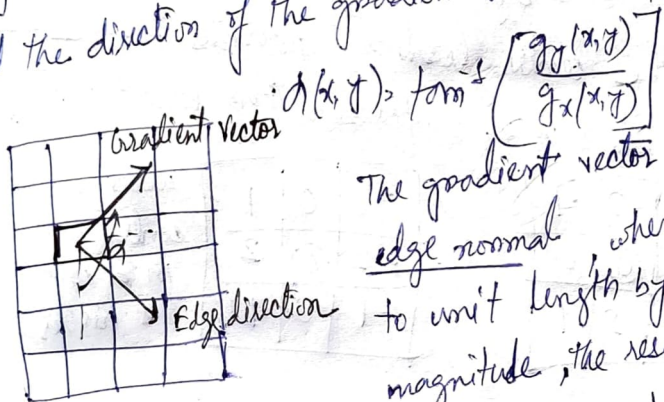
The Image Gradient and its properties:-

$$\nabla f(x,y) = \text{grad}[f(x,y)] = \begin{bmatrix} g_x(x,y) \\ g_y(x,y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

It points in the direction of maximum rate of change of f at (x,y) when applicable for all values of x and y . $\nabla f(x,y)$ becomes a vector

image, whose magnitude is, $M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$

and the direction of the gradient vector at point (x,y) is



The gradient vector sometimes is called edge normal when the vector is normalised to unit length by dividing it by its magnitude, the resulting vector is referred to as the edge unit normal.

Gradient Operators:

$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

$$\begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}$$

$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

$$\begin{bmatrix} 0 & 0 \\ -1 & 1 \end{bmatrix}$$

when diagonal edge direction is of interest, we need 2D-kernels

Robert's cross-gradient operators.

$$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$g_x = \frac{\partial f}{\partial x}(z_0, z_5)$$

$$g_y = \frac{\partial f}{\partial y}(z_5, z_0)$$

for kernels, which are symmetric about their centers, for that we need 3×3 kernel atleast. These kernels, take into account the nature of the data on the opposite sides of the center points thus carry more information regarding direction of an edge.

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Perritt operators

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Sobel's operators

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

using a 2 in the center location provides image smoothing. (to better noise suppression than Perritt operators)

weight of central point in a row is higher

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

weight of central point in a column is higher.

To reduce computational burden, we approximate the magnitude of the gradient by absolute values —

$$i.e. \quad M(x, y) \approx |g_x| + |g_y|$$

It preserves the relative changes in intensity levels, but the trade off is the resulting filters will not be isotropic.

Kirsch Compass kernels

It is designed to detect edge magnitude and direction in all eight compass directions.

-3	-3	5
-3	0	5
-3	-3	5

N

-3	5	5
-3	0	5
-3	-3	-3

NW

6	5	5
-3	0	-3
-3	-3	-3

W

5	5	-3
5	0	-3
-3	-3	-3

SW ...

for eight direction $\rightarrow$ eight orientation.
 apply each at a point and take the one with maximum response.

Max-Heldreth Edge Detector

- ① The change in intensities depends on scale, implying that their detection requires using operators of different sizes. Capable of being 'tuned' at any scale.
- ② At edge points changes occur, so derivative operator is to be used. Kernel must be able to compute 1st and order derivative in discrete domain.

G is a 2D-gaussian function —

$$G(x, y) = e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

$\nabla^2 G \rightarrow$ Laplacian of the gaussian

$$\Delta G = \nabla^2 G(x, y) = \frac{\partial^2 G(x, y)}{\partial x^2} + \frac{\partial^2 G(x, y)}{\partial y^2}$$

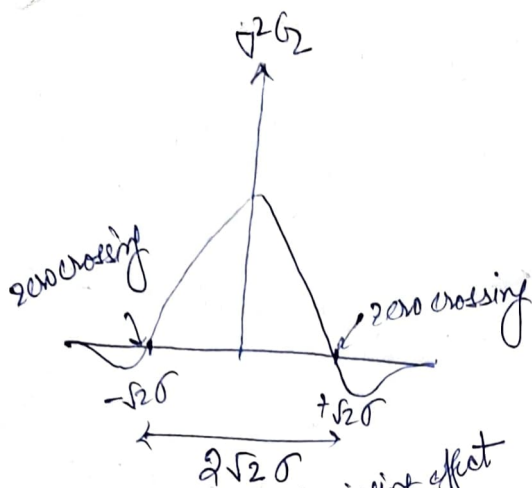
$$= \frac{\partial}{\partial x} \left[\frac{-x}{\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}} \right] + \frac{\partial}{\partial y} \left[\frac{-y}{\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}} \right]$$

$$= \left[\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}} + \left[\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

$$= e^{-\frac{x^2 + y^2}{2\sigma^2}} \left[\frac{x^2}{\sigma^4} + \frac{y^2}{\sigma^4} - \frac{2}{\sigma^2} \right]$$

$$= \frac{e^{-\frac{x^2 + y^2}{2\sigma^2}}}{\sigma^4} [x^2 + y^2 - 2\sigma^2]$$

$\rightarrow$ Circle of radius $\sqrt{2}\sigma$



0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

- The Gaussian part of the operator blurs the image, thus reducing the intensity of structures at scales much smaller than σ . Unlike averaging filter, Gaussian function is smooth in both the spatial and frequency domains, and is likely to introduce artifacts not present in original image.
- Secondly, derivative properties of Laplacian, although first derivatives can be used for detecting abrupt changes in intensity, they are directional operators. And also Laplacian is isotropic.

$$g(x, y) = [\nabla^2 G(x, y)] \star f(x, y)$$

$$\text{OR, } g(x, y) = \nabla^2 [G(x, y) \star f(x, y)]$$

So, the Marr-Hildreth edge-detection algorithm is summarized as —

- (i) Filter the input image with an $n \times n$ Gaussian lowpass kernel obtained by sampling.
- (ii) Compute the Laplacian of the image resulting from step 1.
- (iii) Find the zero crossings of the image from step 2.

zero-crossing detection —

Let P be a point at center in a 3×3 neighbourhood. A zero crossing at P implies that the signs of at least two of its opposing neighbouring pixels must

differ.

0	✓	x
1	P	1
x	✓	0

If the value of $g(x,y)$ are being compared against a threshold, then not only must sign should differ, the absolute value of their difference must also exceed the threshold to be 'p' a zero-crossing pixel.

The Canny Edge Detector

Objective

- Low error rate $\rightarrow$ True edges will not be missed
 $\rightarrow$ False detection of edges to be minimized.

- Edge points should be well localized $\rightarrow$ The edges located must be as close as possible to the true edges, i.e. the distance between a point marked as an edge by the detector and the center of the true-edge should be minimum.

- Single edge-point response $\rightarrow$ The detector should return only one point for each true edge point i.e. there should be one detected edge pixel.

Steps

- Gaussian smoothing (to reduce noise) $f_s(x,y) = f(x,y) \star g(x,y)$
 $g(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$
 $f_s(x,y) = f(x,y) \star g(x,y)$
 $\nearrow$ smoothed image

- Compute gradient magnitude & direction (use any kernel)

$$M_s(x,y) = \|\nabla f_s(x,y)\| = \sqrt{g_x^2 + g_y^2}$$

$$\alpha(x,y) = \tan^{-1} \left[\frac{g_y}{g_x} \right]$$

Gradient image $\|\nabla f_s(x,y)\|$ typically contains wide ridges around local maxima. So next step is to thin these ridges.

- Non-Maxima Suppression

Let d_1, d_2, d_3 and d_4 denote the four basic edge directions, we can formulate the nonmaxima suppression scheme for a 3×3 region centered at an arbitrary point (x,y) in α :-

- (a) Find the direction d_k closest to $\nabla f(x,y)$.
- (b) Let k denote the value of $\|\nabla f_k\|$ at (x,y) . If k is less than the value of $\|\nabla f\|$ at one or both of the neighbors of point (x,y) along d_k , let $g_N(x,y) = 0$ (suppression)

otherwise let $g_N(x,y) = k$

The final operation is to threshold $g_N(x,y)$ to reduce false edge points. In Marr-Hildreth algo, single threshold is used. If we set the threshold too low, there'll still be some false edges (false positives). If the threshold is set too high, then valid edge points will be eliminated (false negatives).
Canny's algo attempts to improve this using hysteresis thresholding.

- (4) Double thresholding to detect & link the edges

A low threshold $\rightarrow T_L$

A high threshold $\rightarrow T_H$

The ratio of the high to low threshold should be in the range of 2:1 to 3:1.

We can visualize the thresholding operation as creating two additional images —

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

Initially,
 $g_{NH}(x,y)$
 $g_{NL}(x,y)$
set to '0'.

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

The nonzero pixels in $g_{NH}(x,y)$ and $g_{NL}(x,y)$ may be viewed as being 'strong' and 'weak' edge pixels. After thresholding, all strong pixels in $g_{NH}(x,y)$ are

assumed to be valid edge pixels, and are so marked immediately. Depending on the value of T_H , the edges in $g_{NH}(x,y)$ typically have gaps. Longer edges are formed using the following procedure —

- (a) Locate the next unvisited edge pixel, P , in $g_{NH}(x,y)$
- (b) Mark as valid edge pixels, all the weak pixels in $g_{NL}(x,y)$ that are connected to P using 8-connectivity.
- (c) If all nonzero pixels in $g_{NH}(x,y)$ have been visited go to step (d), else return to step (a).
- (d) Set to zero all pixels in $g_{NL}(x,y)$ that were not marked as valid edge pixels.

At the end, the final image output by the Canny's algo is formed by appending to $g_{NH}(x,y)$ all the nonzero pixels from $g_{NL}(x,y)$.

We can go further for local linking.

For every so far detected edge points, if there is discontinuities look into its neighborhood w.r.t. certain properties —

$$(a) |M(s,t) - M(x,y)| \leq E \rightarrow \text{Magnitude}$$

$$(b) |\alpha(s,t) - \alpha(x,y)| \leq A \rightarrow \text{direction}$$

(either one or both)

Hough Transform — We only have set of edge pixels but which pixel belongs to which object $\rightarrow$ not known, no local info is known —

Global Processing All pixels are candidates for linking, and have to be accepted or eliminated based on predefined global properties. Whether a set of pixels lie on curves of ~

specified shape. Once detected, these curves form the edges on region boundaries of interest.

For straight line detection,

(x_i, y_i) be a pixel

$$y_i = a_i x_i + b_i \text{ (slope intercept form)}$$

With different (a_i, b_i) infinite no. of lines may pass through (x_i, y_i)

Another point $(x_j, y_j) \rightarrow$

$$y_j = a_j x_j + b_j$$

$$y_i = a_i x_i + b_i \rightarrow b_i = -a_i x_i + y_i \text{ [in a-b plane]}$$

slope $\rightarrow -x_i$
intercept y_i

(x, y) plane $\Rightarrow$ A line $\rightarrow$ no. of points (x_i, y_i) on the line (a_i', b_i')

for all such points, lines in parameter space will intersect at (a_i', b_i')

A [slope] [intercept] $\rightarrow$ initially all zero.

for each point (x_i, y_i)
for each possible value of a_i
find $b_i = -a_i x_i + y_i$
increment $A[a_i][b_i]++$

OR
for each point (x_i, y_i)
for each possible value of b_i
find $a_i = (y_i - b_i) / x_i$
 $A[a_i][b_i]++$

Apply the threshold on $A[a_i][b_i]$

those that'll survive will form the lines.
In the acc, along with count, keeps $x_{max}, x_{min}, y_{max}, y_{min}$ to recognize segments.

Region Extraction

Pixel Based Approach

Region based approach

Pixel Based Approach:

feature/property thresholding $\rightarrow$ selection of feature
 $\rightarrow$ selection of threshold

- $\rightarrow$ For every pixel calculate the feature
 - $\rightarrow$ Study the feature distribution in feature space.
 - $\rightarrow$ For different types of regions different groups/cluster will be formed in the feature space.
 - $\rightarrow$ Find out the discriminating boundary to isolate the clusters.
- distribution of a feature can be approximated as histogram and the boundary function to a threshold.

Two types of Regions $\rightarrow$

$$b(x, c) = \begin{cases} 1 & \text{if } p(x, c) \leq t \\ 0 & \text{if } p(x, c) > t \end{cases}$$

$t = \text{threshold}$
 $p(x, c) \rightarrow \text{feature value}$

Choice of Feature $\leftarrow$ (very important for segmentation)



$\rightarrow$ background (127)
 $\rightarrow$ foreground (255)



$\rightarrow 127$
 $\rightarrow 255, 255$
 $\rightarrow 50\%, 191$
 $\rightarrow 255, 127$
 $\rightarrow$ mean filter
 $\rightarrow \approx 191$ threshold



$\rightarrow 255, 0$
 $\rightarrow 50\%, 127$
 $\rightarrow 255, 255$
 $\rightarrow$ uniformly distributed
 $\rightarrow$ gradient threshold

Intensity $\rightarrow$ feature
 threshold, $t = 127$ to 254

Selection of Threshold (based on histogram)

(No. of regions) - 1 = No. of thresholds.

Image $f(x, y)$ of size $M \times N$ has two types of regions $\rightarrow$ 1 threshold. Max feature value $\rightarrow L-1$

feature histogram $\rightarrow h$

If feature value of one region is high i.e. no. of pixels in one region is p% of total no. of pixels in the image.

Therefore threshold t such that -

$$\sum_{i=t+1}^{L-1} h[i] = \frac{PMN}{100}$$

[Can't be applied where P is **not** known]

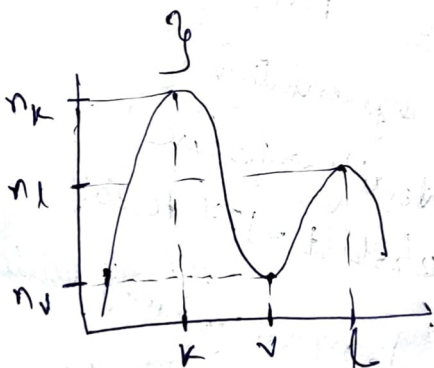
smallest value of t is such that

$$\sum_{i=t+1}^{L-1} h[i] \geq \frac{PMN}{100}$$

Sum = 0

for $(i = L-1; i \geq t; i--)$

{ sum += h[i]
if (sum > $\frac{PMN}{100}$)
break;



2 types of Regions
Bimodal $\rightarrow$ Modes denote frequently occurring property values.

Two distinct peaks/modes at $z=k$ and $z=l$.

deep valley at $z=v$ between the peaks giving the histogram a bimodal shape. let say s_1 and s_2 are magnitude of slopes of lines joining bottom of valley and each of the peaks i.e.

$$s_1 = \left| \frac{n_k - n_v}{k - v} \right|$$

$$s_2 = \left| \frac{n_l - n_v}{l - v} \right|$$

Therefore, the histogram is said to —

(i) be bimodal if $s_1 > th$ and $s_2 > th$

(ii) have peak and shoulder if $s_1 > th$ and $s'_2 \leq th$ or vice versa

(iii) be flat if $s_1 \leq th$ and $s_2 \leq th$

$th \rightarrow$ predefined threshold.

the strength of the bimodality, $S_b = s_1 s_2$

Bimodal

$\rightarrow$ strongly

$\rightarrow$ weakly

$\rightarrow$ because of noise & blurring deep valley may not be found.

$\rightarrow$ overlap in feature values between the regions

$\rightarrow$ Boundary of the regions may overlap rather than the object interior.

Discrete values $\rightarrow$ Non continuous histogram $\rightarrow$ jaggedness

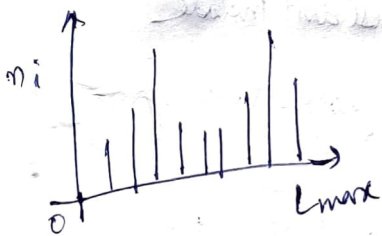
finding the valley is non-trivial.



Valley Detection:-

$\rightarrow$ Apply mean filter

$$\bar{n}_i = \frac{1}{2q+1} \sum_{j=i-q}^{i+q} n_j$$



$$\bar{n}_i = \bar{n}_i$$

$\rightarrow$ find local maxima in a $2q+1$ window at $z_1, z_2, \dots, z_k$

$$n_{z_i} = \max \{n_i\} \quad z_i - s \leq i \leq z_i + s$$

$\rightarrow$ find the valley $n_v, n_v \leq n_i \quad \forall i$

$\rightarrow$ find $\max \{n_{z_i}\}$ for $z_1 < v$

$\rightarrow$ find $\max \{n_{z_i}\}$ for $z_k > v$

Threshold Selection

Global technique (simple)

1. $Th = \mu$ (avg intensity)

2. If $f(x, y) > Th$ then $f(x, y) \in \text{class 1}$
else $f(x, y) \in \text{class 2}$

3. Calculate μ_1 and μ_2 (avg intensity of class 1 & class 2 respectively)

4. If $Th - (\mu_1 + \mu_2)/2$ is small then STOP

else $\{ Th = (\mu_1 + \mu_2)/2 \}$ go to step 2

Region Based Approach:-

① Region Growing:-

① Consider a seed pixel (initial Region)

② Check whether the inclusion of neighbour pixel (4/8) contains the homogeneity of property

If yes, include the pixel in the region else don't include

③ For newly included pixel check its neighbours recursively for possible inclusion.

Once recursion stops find another seed pixel if all the pixels are not included $\rightarrow$ one region.

Influencing factors $\rightarrow$ (i) Selection of seed pixel
(ii) Selection of property

Property $\rightarrow$ Variance of intensity in the region $< Th \rightarrow$ As a region grows

$$[\max\{f(x, y)\} - \min\{f(x, y)\}] < Th$$

they allow more variation.

Region Splitting :- (Top Down Approach)

Rectangular homogeneous regions on a rectangular region $\rightarrow$ check the property
If homogeneity is not satisfied then, split into 4 equal rectangle (quadtree)

- (a) Consider whole image as a single region
- (b) Check the property. if not true split it into 4 parts
- (c) Repeat on each rectangles

Size : Row, Col (Power of 2), leaves of quadtree $\rightarrow$ regions.

Region Merging :- (Bottom up approach)

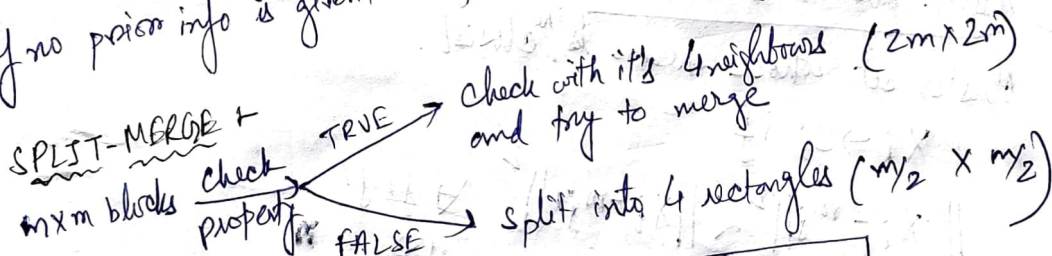
$\rightarrow$ Starts with individual pixels as individual regions
 $\rightarrow$ check 2×2 neighbouring regions $\rightarrow$ whether they satisfy the property together, if so merge

leaves $\rightarrow$ once merged higher level node is formed
 $PROP(R_i \cup R_j) = \text{FALSE}$ where R_i, R_j adjacent

Regions are large $\rightarrow$ split

Regions are small $\rightarrow$ merge

If no prior info is given, then split and merge.



split / Merge / Split-Merge $\rightarrow$ Region formation $\rightarrow$ Region Adjacency Graph

(i) Each detected region is a node in the graph

(ii) Regions which are adjacent (Common Boundary) corresponding nodes will be connected by edge.

(iii) R_i and R_j are two connected nodes for each pair of connected nodes (R_i, R_j) check $PROP(R_i \cup R_j)$

If false then delete the edge.

Each subgraph $\rightarrow$ maximally connected regions

Region Segmentation using K-Means clustering:-

Let N observations $\rightarrow \{X_1, X_2, \dots, X_N\}$

Each observation is of the form, X_i is represented by n -dimensional vector,

$$X_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix}^T$$

one pixel $\rightarrow$ observation $\rightarrow$ n dimensional vector $\rightarrow$ feature vector

For RGB color, X_i typically is a 3-D vector

N observations are partitioned into disjoint k groups/clusters $(C_1, C_2, \dots, C_k)$, so that the following criterion is

satisfied —

$$\arg \min_c \left(\sum_{i=1}^k \sum_{X \in C_i} \|X - m_i\|^2 \right)$$

where $m_i \rightarrow$ mean vector / centroid of cluster C_i

Algorithm

① Initialization • Initialize k -cluster centers, m_i $[i=1, 2, \dots, k]$
A may be imaginary, randomly pick k observations

② Assign observations to clusters • Assign each sample to the cluster set whose mean is the closest.

$$X_i \rightarrow C_j \text{ if}$$

$$\|X_i - m_j\|^2 < \|X_i - m_{j'}\|^2 \quad \forall j \neq j' \\ i=1, 2, \dots, N \\ j, j'=1, 2, \dots, k$$

③ Update the cluster centers (means) •

$$m_i = \frac{1}{|C_i|} \sum_{X \in C_i} X \quad i=1, 2, \dots, k$$

where, $|C_i|$ is the no. of observations in cluster set C_i

④ Test for Completion •

① $\rightarrow$ When no sample is changing clusters in successive iterations.

- (i) Compute the Euclidean norm of the difference between the centroids in the current and previous steps, Compute Residual error, E as the sum of the k norms,
 Stop if $E \leq Th$ [$Th \rightarrow$ non-negative threshold]

Else go back to step (2)

Super-Pixel Based Segmentation

Supapixel $\rightarrow$ Replace the standard pixel grid by grouping pixels into primitive regions that are more perceptually meaningful than the individual pixels.

- Advantages $\rightarrow$
- (i) Supapixel is perceptually more meaningful
 - (ii) lead to less computation.
 - (iii) Put less detailing on the details.

$N_{sp} \Rightarrow$ no. of superpixels desired

$N_p \Rightarrow$ No. of pixels in the image.

Image is partitioned into equisized N_{sp}
 $\downarrow$
 no. of superpixels ($S \times S$)

$$\text{no. of superpixel} = \frac{N_p}{N_{sp}}$$

$$S = \left[\frac{N_p}{N_{sp}} \right]^{1/2}$$

for each pixel in RGB color system, 5-D vector has the form

$$Z = [r \ g \ b \ x \ y]^T$$

SLIC (Simple Linear Iterative Clustering)

- (1) Initialize the clusters + Each superpixel is the cluster, i
 Compute initial superpixel cluster centers,

$$m_i = [r_i \ g_i \ b_i \ x_i \ y_i]^T \quad i = 1, 2, \dots, N_{sp}$$

by sampling the image at regular grid step, s . Move the cluster centers to the lowest gradient position in a 3×3 neighbourhood. For each pixel location, p in the image, set a label, $L(p) = -1$ and a distance, $d(p) = \infty$

② Assign samples to cluster centers

For each cluster/supapixel, compute the distance $D_i(P)$ between m_i and each pixel P in a $2s \times 2s$ neighbourhood about m_i .

Then, for each P and $i=1, 2, \dots, N_{sp}$

if $D_i < d(P)$

{ $L(P) = i$

$d(P) = D_i$

}

③ update the cluster centers let C_i denote the set of pixels in the image with label $L(P)=i$, update m_i as —

$$m_i = \frac{1}{|C_i|} \sum_{P \in C_i} P \quad i=1, 2, \dots, N_{sp}$$

④ Test for Convergence : If ECT goto step ⑤ else go back to step ②

⑤ Post-process the supapixel regions Replace all the superpixels in each region, C_i by their avg values, m_i . d_c and d_s denote the color and spatial euclidean distances between two points in a cluster —

$$d_c = \sqrt{(r_j - r_i)^2 + (g_j - g_i)^2 + (b_j - b_i)^2}$$

$$d_s = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

define D as composite distance —

$$D = \left[\left(\frac{d_c}{d_{cm}} \right)^2 + \left(\frac{d_s}{d_{sm}} \right)^2 \right]^{1/2}$$

d_{cm} and d_{sm} are more expected values of d_c and d_s .